From Control to Offline Reinforcement Learning

Learning Better Decisions from Logged Data

Stephan Mandt

June 30, 2026

Predictions Are Not Decisions

Supervised learning predicts	RL chooses
Truck arrival time	Which truck to unload next
Unloading duration	Which pit to assign
Grain quality	Which silo to route into
Train delay	How to schedule train loading

Business takeaway

Supervised learning predicts what happens next. Reinforcement learning chooses what should happen next.

Actions Change the Future

Choosing a truck, pit, route, or silo changes future queues, storage, blending options, and train schedules.

Product relevance

In RL, the model does not just observe the data distribution. It helps create the next data distribution.

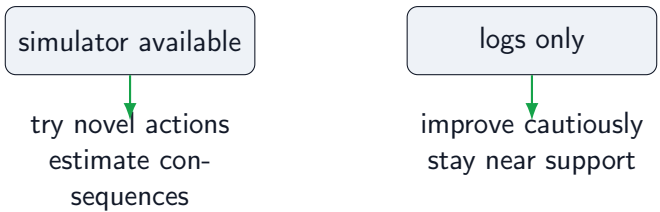
States, Actions, and Policies

Object	Grain-terminal example
state s_t	current queue, silo inventory, grain quality, train progress
action a_t	unload truck, choose pit, route conveyor, assign silo, fill car
policy $\pi(a \mid s)$	rule or model that chooses actions from states
next state s_{t+1}	the facility after the action has changed it

Business takeaway

A decision problem starts with states, actions, and a policy that maps situations to decisions.

Why RL Needs More Than Logs



Business takeaway

RL is about policy improvement, not just action prediction.

The RL Question

Given logged grain-terminal decisions and delayed outcomes, can we learn a better policy without risky online experimentation?

Logged data:

$$(s_t, a_t, \text{outcome}, s_{t+1})$$

Product relevance

This is the bridge from product operations to imitation learning and the RL formalism.

Imitation Learning: Turn Decisions Into Supervised Learning

Given this situation, what would the operator usually do?

$$\mathcal{D} = \{(s_t, a_t)\}, \qquad \tau = (s_1, a_1, \ldots, s_T, a_T)$$

$$\mathcal{L}_{\mathrm{BC}}(\theta) = -\mathbb{E}_{(s,a)\sim\mathcal{D}} \left[\log \pi_\theta(a \mid s)\right]$$

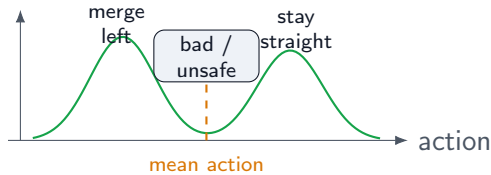
For deterministic continuous actions, use regression:

$$\min_\theta \ \mathbb{E}\left[\|a - \pi_\theta(s)\|^2\right]$$

Product relevance

Attractive because it is fully offline, simple to train, and does not require defining a reward function.

Pitfall 1: Averaging Demonstrations Can Fail



Consider imitation data for self-driving cars:

- in similar observed states, driver 1 stays in lane
- driver 2 changes lanes
- squared-error regression learns the conditional mean action

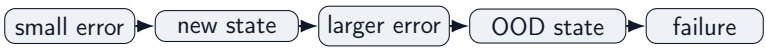
Problem: the mean action can lie between valid modes: a poor command that no expert demonstrated.

Pitfall 2: Small Mistakes Compound

In supervised learning, errors do not change the next input.

In behavior cloning, actions change future states:

$$p_{\text{expert}}(s) \neq p_{\pi}(s)$$



One classical fix is DAgger: roll out the learned policy, ask the expert for corrections, and aggregate the new data.

Product relevance

Offline logs often underrepresent recovery states, rare failures, and near-miss conditions.

Transition: Behavior cloning asks what action was likely in the logs. RL asks which action led to better long-term outcomes.

From Decisions to MDPs

A Markov Decision Process adds dynamics and rewards:

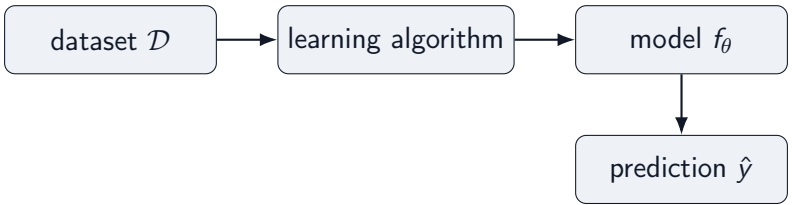
$$(\mathcal{S}, \mathcal{A}, P, r, \gamma)$$

- $\mathcal{S}$: possible states
- $\mathcal{A}$: possible actions
- $P(s' \mid s, a)$: transition
- $r(s, a)$: reward
- $\gamma \in [0, 1)$: discount

Product relevance

The key modeling assumption is that s_t contains enough information to make the next decision.

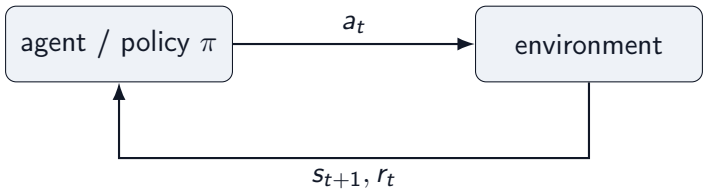
Supervised Learning Setup



$$\mathcal{D} = \{(x_i, y_i)\}_{i=1}^{n}, \qquad \theta^\star = \arg\min_\theta \sum_i \ell(f_\theta(x_i), y_i)$$

The model predicts labels from a fixed dataset.

Agent-Environment Loop



- **Agent:** chooses the next action from the current state.
- **Environment:** the facility or simulation that returns the next state and reward.
- **Key contrast:** supervised learning predicts from fixed data; RL actions create the next data.

What RL Adds

Challenges

- actions change future states
- rewards can be delayed
- errors compound over time
- exploration can be risky

Opportunities

- optimize decisions, not labels
- use simulation to try new actions
- improve long-term outcomes
- learn beyond imitation

Business takeaway

Offline RL is the version closest to supervised learning: learn from a fixed dataset, but optimize a policy.

Value and Discounting

The value of a state is expected discounted reward:

$$V^{\pi}(s) = \mathbb{E}_{\pi}\left[\sum_{k=0}^{\infty} \gamma^{k} r_{t+k} \,\middle|\, s_t = s\right]$$

The discount factor $0 < \gamma < 1$ controls how much the agent plans ahead.

- small γ: more myopic, closer to greedy behavior
- large γ: more weight on delayed consequences
- finance analogy: like net present value, rewards today count more than equal future rewards
- grain loading: a greedy fill policy optimizes each railcar; a farsighted policy optimizes the entire train

V, Q, and Advantage

$$Q^\pi(s,a) = \text{value of taking } a \text{ in } s, \text{ then following } \pi$$
$$A^\pi(s,a) = Q^\pi(s,a) - V^\pi(s)$$

Business takeaway

Advantage asks: was this action better or worse than the policy's usual action in the same state?

Bellman Thinking

$$Q(s, a) \approx r(s, a) + \gamma V(s')$$

The value of an action is immediate reward plus the value of where it takes us.

Product relevance

Bellman recursion is the RL version of cost-to-go recursion in optimal control.

Excursion: Control and RL

Now that we have RL notation, classical control is easy to place:

$$\text{controller} \approx \text{policy}, \qquad \text{cost-to-go} \approx \text{value function}.$$

Both fields ask how to choose actions that improve future outcomes.

Product relevance

In the grain terminal, both a hand-designed controller and an RL policy are decision rules for routing, assignment, and loading.

Control $\leftrightarrow$ RL Dictionary

Control theory	**Reinforcement learning**
State x_t	State s_t
Control u_t	Action a_t
Dynamics f	Transition $P(s' \mid s, a)$
Cost $c(x, u)$	Negative reward $-r(s, a)$
Controller	Policy $\pi(a \mid s)$
Cost-to-go $J(x)$	Value function $V(s)$
Dynamic programming / HJB	Bellman equations
MPC	Model-based RL / planning

Classical Example: PID Control

A PID controller chooses an action from tracking error:

$$u_t = K_P e_t + K_I \sum_{\tau \leq t} e_\tau + K_D (e_t - e_{t-1})$$

It is simple, interpretable, and often very strong when the control objective is local and well specified.

PID as an RL Policy

Define the RL state to include the PID features:

$$s_t = (y_t,\, e_t,\, I_t,\, D_t)\,, \quad I_t = \sum_{\tau \le t} e_\tau, \quad D_t = e_t - e_{t-1}.$$

Then PID is a deterministic policy:

$$a_t = \pi_K(s_t) = K_P e_t + K_I I_t + K_D D_t, \qquad K = (K_P, K_I, K_D).$$

The environment remains the transition model:

$$s_{t+1} \sim p(s_{t+1} \mid s_t, a_t).$$

- Since K parameterizes a policy, RL can tune K by optimizing return.

Business takeaway

PID is a structured policy class; RL can tune its parameters by optimizing return.

PID and RL: Different Strengths

Classical / PID control	Reinforcement learning
Simple and interpretable	Optimizes delayed objectives
Strong local feedback	Handles sequential tradeoffs
Easier to validate	Learns from data and simulation
Needs hand-designed structure	Can adapt to nonlinear operations
Usually optimizes a proxy	Can optimize the KPI directly

Business takeaway

Classical control is often the right starting point. RL becomes attractive when the best action depends on long-horizon operational consequences.

Why RL Should Improve Long Term

- It can optimize delayed outcomes, not only immediate tracking error.
- It can use simulators to evaluate actions not yet tried in production.
- It can learn terminal-specific interactions among queues, silos, conveyors, and trains.
- It can update as operating conditions and objectives change.

Product relevance

For grain loading, the best local move may not be the best move for throughput, quality, rehandling, and train delay.

Online RL Loop

Online RL learns by interacting while training.

Three Common RL Families

Family	Basic idea
Value-based RL	Learn $Q(s, a)$, act greedily
Policy gradient	Directly improve $\pi(a \mid s)$
Actor-critic	Learn both policy and value

Why Online RL Is Hard in Products

- Unsafe or costly exploration
- Customer-facing experiments
- Long feedback delays
- Sparse or noisy rewards
- Operational constraints and compliance

Product relevance

When exploration is expensive, risky, or product-constrained, logged data becomes the natural starting point.

Model-Based RL: Learn the Dynamics, Then Plan

$$\hat{s}_{t+1} = \hat{f}_\theta(s_t, a_t)$$

$$a^\star_{0:H} = \arg\max_{a_{0:H}} \sum_{t=0}^{H} \gamma^t \hat{r}(s_t, a_t)$$

Business takeaway

Model-based RL often looks like MPC with a learned dynamics model.

Risks: model bias, uncertainty, and compounding rollout error.

Offline RL Setting

Fixed dataset:

$$\mathcal{D} = \{(s_t, a_t, r_t, s_{t+1})\}$$

Behavior policy:

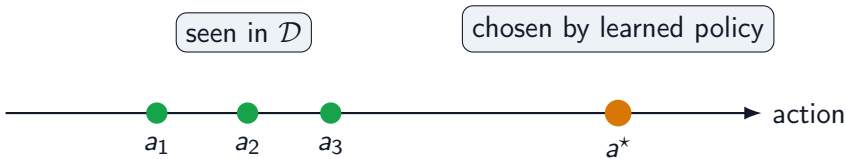
$$\mu(a \mid s)$$

Goal: learn $\pi(a \mid s)$ without online interaction during training.

Product relevance

Logged data source: terminal operator logs. Action space: truck selection, pit assignment, conveyor route, silo, and loading schedule.

Support Mismatch



The policy may choose an unsupported action where the value estimate is unreliable.

Why Naive Q-Learning Fails Offline

Dangerous backup:

$$\max_a Q(s, a)$$

The max can select actions that were rarely or never observed.

Then the learned policy may exploit errors in Q outside the data distribution.

The Offline RL Thesis

Offline RL is mostly about preventing the policy from exploiting value errors outside the data distribution.

The rest of the talk builds a ladder:

$$\text{BC} \rightarrow \text{AWR} \rightarrow \text{IQL} \rightarrow \text{IDQL}$$

Behavior Cloning

$$\mathcal{L}_{\mathrm{BC}}(\theta) = -\mathbb{E}_{(s,a)\sim\mathcal{D}}\left[\log \pi_\theta(a \mid s)\right]$$

Learn to imitate the actions in the dataset.

Benefits: stable, simple, scalable, no explicit value learning.

BC Clones All Actions Equally

BC does not know which actions were good. It clones good, mediocre, and bad actions alike.

Advantage-Weighted Regression

$$\mathcal{L}_{\mathrm{AWR}}(\theta) = -\mathbb{E}_{(s,a)\sim\mathcal{D}}\left[w(s,a)\log \pi_\theta(a \mid s)\right]$$

$$w(s,a) \propto \exp\left(\frac{A(s,a)}{\lambda}\right), \qquad A(s,a) = Q(s,a) - V(s)$$

BC vs. AWR Weighting

High-advantage actions are cloned more strongly; low-advantage actions are downweighted.

AWR Is the First RL Step

AWR is the smallest conceptual step from imitation learning to reinforcement learning.

It still looks like supervised learning, but the labels are now weighted by long-term value.

Product relevance

If logs contain good and bad decisions, AWR gives a way to imitate the better ones more.

IQL Critic Learning

Avoid explicit maximization over unseen actions:

$$\text{avoid: } \max_a Q(s, a)$$

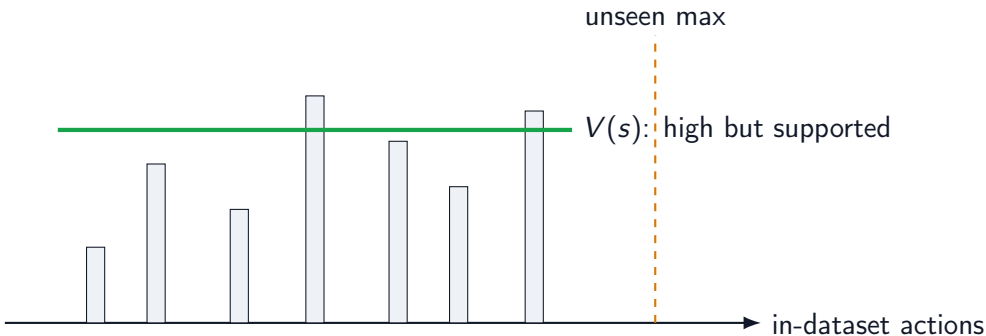
Dataset transition update:

$$Q(s, a) \leftarrow r + \gamma V(s')$$

Learn V as an upper expectile of in-dataset Q values:

$$\mathcal{L}_V = \mathbb{E}_{(s,a)\sim\mathcal{D}} \left[\left| \tau - 1_{Q(s,a)-V(s)<0} \right| (Q(s, a) - V(s))^2 \right], \quad \tau > 0.5$$

In-Sample Value Learning



IQL estimates a high value from in-dataset actions instead of maximizing over all actions.

IQL Policy Extraction

$$\mathcal{L}_\pi = -\mathbb{E}_{(s,a)\sim\mathcal{D}}\left[\exp\left(\frac{Q(s,a)-V(s)}{\beta}\right)\log\pi_\theta(a \mid s)\right]$$

IQL uses advantage-weighted policy extraction after in-sample value learning.

IDQL: When the Action Distribution Is Multimodal

A simple Gaussian policy may average two good actions into a bad one.

Diffusion Policies: Generate Actions by Denoising

$p_\psi(a_{k-1} \mid a_k, s)$

A diffusion policy represents $p_\psi(a \mid s)$ as an iterative conditional generation process [1].

Instead of predicting one mean action, it can sample diverse plausible actions.

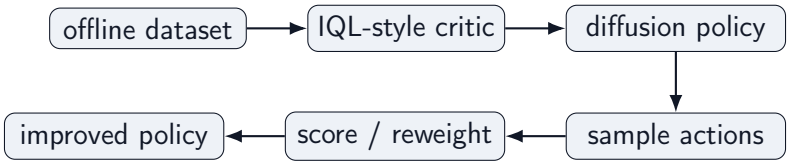
Why Diffusion Policies Fit Offline RL

- They can model multimodal action distributions.
- They stay anchored to logged behavior through behavior-model training.
- They expose multiple candidate actions for critic scoring.
- They are useful when action spaces are continuous, structured, or high-dimensional.

Business takeaway

Diffusion policies separate representation from improvement: sample plausible actions first, then use value estimates to prefer better ones.

IDQL Pipeline



IDQL combines an IQL-style critic with diffusion-based policy extraction [2].

Business takeaway

IQL tells us which actions are valuable; IDQL gives us a richer way to represent and sample them.

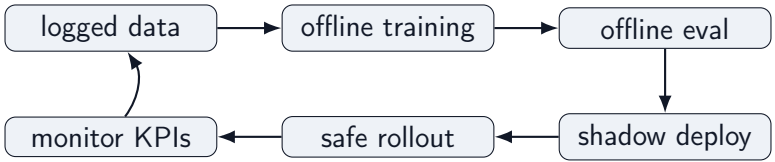
What Matters in Real Offline RL Systems?

1. **State quality:** is the decision approximately Markovian?
2. **Action support:** does $\mathcal{D}$ cover actions we want π to take?
3. **Reward design:** does reward reflect throughput, quality, delay, rehandling, and train-loading penalties?
4. **Counterfactual risk:** are we evaluating rare or unseen actions?
5. **Evaluation:** offline evaluation, simulation, shadow deployment, or cautious rollout?
6. **Policy constraints:** how far may π move from μ?

Product relevance

Safe deployment strategy: shadow recommendations, operator approval, and constrained rollout.

Product Architecture Sketch



Closing Thesis

RL is data-driven optimal control.

Offline RL is controlled policy improvement from logged data.

The ladder:

$$\text{control} \to \text{RL} \to \text{offline RL} \to \text{BC} \to \text{AWR} \to \text{IQL} \to \text{IDQL}$$

References

Cheng Chi, Zhenjia Xu, Siyuan Feng, Eric Cousineau, Yilun Du, Benjamin Burchfiel, Russ Tedrake, and Shuran Song.
Diffusion policy: Visuomotor policy learning via action diffusion.
arXiv preprint arXiv:2303.04137, 2023.

Philippe Hansen-Estruch, Ilya Kostrikov, Michael Janner, Jingyun G. Chen, and Sergey Levine.
IDQL: Implicit q-learning as an actor-critic method with diffusion policies.
arXiv preprint arXiv:2304.10573, 2023.

Ilya Kostrikov, Ashvin Nair, and Sergey Levine.
Offline reinforcement learning with implicit q-learning.
arXiv preprint arXiv:2110.06169, 2021.

Sergey Levine, Aviral Kumar, George Tucker, and Justin Fu.
Offline reinforcement learning: Tutorial, review, and perspectives on open problems.
arXiv preprint arXiv:2005.01643, 2020.

Xue Bin Peng, Aviral Kumar, Grace Zhang, and Sergey Levine.
Advantage-weighted regression: Simple and scalable off-policy reinforcement learning.
arXiv preprint arXiv:1910.00177, 2019.

Richard S. Sutton and Andrew G. Barto.
Reinforcement Learning: An Introduction.
MIT Press, 2 edition, 2018.